

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



ADVANCE CRYPTOGRAPHY AND CODING THEORY
(CO3083)

Assignment

Advisor

Group

Students Nguyễn Tấn Tài 2212990

Nguyễn Chánh Tín 2213491

.....

HO CHI MINH CITY, OCTOBER 2025

Mục lục

1	a	1
1.1	Các khái niệm cơ bản	1
1.1.1	Không gian vector (Vector Spaces)	1
1.1.2	Tổ hợp tuyến tính (Linear Combinations)	1
1.1.3	Độc lập tuyến tính (Independence)	1
1.1.4	Cơ sở (Bases)	2
1.1.5	Cơ sở trực giao và trực chuẩn (Orthogonal and Orthonormal Basis)	2
1.1.6	Lattice (Mạng tinh thể)	2
1.1.7	Miền cơ bản (Fundamental Domain)	3
1.1.8	Quả cầu Euclide (The Euclidean Ball)	3
1.1.9	Bài toán vector ngắn nhất (The Shortest Vector Problem - SVP)	3
1.1.10	Bài toán vector gần nhất (The Closest Vector Problem - CVP)	3
1.1.11	Độ dài ngắn nhất kỳ vọng theo Gauss (The Gaussian Expected Shortest Length)	4
1.1.12	Thuật toán LLL (The LLL Algorithm)	4
1.2	Giải quyết bài toán Tìm Đa Thức Tối Thiểu (Minimal Polynomial Finder)	6
1.2.1	Đề bài	6
1.2.2	Phân tích	6
1.2.3	Lý thuyết	6
1.2.4	Phương pháp	8
1.2.5	Thực thi	9
1.2.6	Kiểm thử	13

1 a

1.1 Các khái niệm cơ bản

1.1.1 Không gian vector (Vector Spaces)

Định nghĩa (Không gian vector): Một không gian vector (vector space) trên trường $\mathbb{F}$ là một tập hợp V cùng với hai phép toán:

- Phép cộng vector: $+: V \times V \rightarrow V$
- Phép nhân vô hướng: $\cdot: \mathbb{F} \times V \rightarrow V$

thỏa mãn các tiên đề sau với mọi $\mathbf{u}, \mathbf{v}, \mathbf{w} \in V$ và $a, b \in \mathbb{F}$:

1. $\mathbf{u} + \mathbf{v} = \mathbf{v} + \mathbf{u}$ (tính giao hoán)
2. $(\mathbf{u} + \mathbf{v}) + \mathbf{w} = \mathbf{u} + (\mathbf{v} + \mathbf{w})$ (tính kết hợp)
3. Tồn tại vector không $\mathbf{0} \in V$ sao cho $\mathbf{v} + \mathbf{0} = \mathbf{v}$
4. Với mọi $\mathbf{v} \in V$, tồn tại $-\mathbf{v} \in V$ sao cho $\mathbf{v} + (-\mathbf{v}) = \mathbf{0}$
5. $a(b\mathbf{v}) = (ab)\mathbf{v}$
6. $1\mathbf{v} = \mathbf{v}$
7. $a(\mathbf{u} + \mathbf{v}) = a\mathbf{u} + a\mathbf{v}$
8. $(a + b)\mathbf{v} = a\mathbf{v} + b\mathbf{v}$

Ví dụ: Không gian Euclide n chiều $\mathbb{R}^n$ với phép cộng và nhân vô hướng thông thường là một không gian vector trên trường số thực $\mathbb{R}$.

1.1.2 Tổ hợp tuyến tính (Linear Combinations)

Định nghĩa (Tổ hợp tuyến tính): Cho V là không gian vector trên trường $\mathbb{F}$ và $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n \in V$. Một **tổ hợp tuyến tính** (linear combination) của các vector này là một vector có dạng:

$$\mathbf{v} = a_1\mathbf{v}_1 + a_2\mathbf{v}_2 + \dots + a_n\mathbf{v}_n$$

trong đó $a_1, a_2, \dots, a_n \in \mathbb{F}$ được gọi là các hệ số.

1.1.3 Độc lập tuyến tính (Independence)

Định nghĩa (Độc lập tuyến tính): Một tập hợp các vector $\{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n\}$ trong không gian vector V được gọi là **độc lập tuyến tính** (linearly independent) nếu phương trình:

$$a_1\mathbf{v}_1 + a_2\mathbf{v}_2 + \dots + a_n\mathbf{v}_n = \mathbf{0}$$

kéo theo $a_1 = a_2 = \dots = a_n = 0$.

Ngược lại, nếu tồn tại các hệ số $a_1, \dots, a_n$ không đồng thời bằng 0 sao cho tổ hợp tuyến tính trên bằng $\mathbf{0}$, thì tập vector đó được gọi là **phụ thuộc tuyến tính** (linearly dependent).

1.1.4 Cơ sở (Bases)

Định nghĩa (Cơ sở của không gian vector): Một tập hợp các vector $\mathcal{B} = \{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n\}$ được gọi là một **cơ sở** (basis) của không gian vector V nếu:

1. $\mathcal{B}$ độc lập tuyến tính
2. Mọi vector $\mathbf{v} \in V$ đều có thể biểu diễn duy nhất dưới dạng tổ hợp tuyến tính của các vector trong $\mathcal{B}$

Số lượng vector trong một cơ sở được gọi là **số chiều** (dimension) của không gian vector.

1.1.5 Cơ sở trực giao và trực chuẩn (Orthogonal and Orthonormal Basis)

Định nghĩa 1 (Tích vô hướng). Trong không gian Euclide $\mathbb{R}^n$, tích vô hướng (inner product hoặc dot product) của hai vector $\mathbf{u} = (u_1, \dots, u_n)$ và $\mathbf{v} = (v_1, \dots, v_n)$ được định nghĩa là:

$$\langle \mathbf{u}, \mathbf{v} \rangle = u_1 v_1 + u_2 v_2 + \dots + u_n v_n$$

Định nghĩa 2 (Cơ sở trực giao). Một cơ sở $\mathcal{B} = \{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n\}$ của không gian vector V được gọi là **cơ sở trực giao** (orthogonal basis) nếu:

$$\langle \mathbf{v}_i, \mathbf{v}_j \rangle = 0 \quad \text{với mọi } i \neq j$$

Định nghĩa 3 (Cơ sở trực chuẩn). Một cơ sở trực giao $\mathcal{B} = \{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n\}$ được gọi là **cơ sở trực chuẩn** (orthonormal basis) nếu ngoài tính trực giao, mỗi vector có độ dài bằng 1:

$$\|\mathbf{v}_i\| = \sqrt{\langle \mathbf{v}_i, \mathbf{v}_i \rangle} = 1 \quad \text{với mọi } i$$

Tương đương:

$$\langle \mathbf{v}_i, \mathbf{v}_j \rangle = \delta_{ij} = \begin{cases} 1 & \text{nếu } i = j \\ 0 & \text{nếu } i \neq j \end{cases}$$

Ví dụ: Cơ sở chuẩn tắc (standard basis) của $\mathbb{R}^3$ là:

$$\mathbf{e}_1 = (1, 0, 0), \quad \mathbf{e}_2 = (0, 1, 0), \quad \mathbf{e}_3 = (0, 0, 1)$$

là một cơ sở trực chuẩn.

1.1.6 Lattice (Mạng tinh thể)

Định nghĩa 4 (Lattice). Cho $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n \in \mathbb{R}^m$ là các vector độc lập tuyến tính. **Lattice** L sinh bởi các vector này, ký hiệu $L = L(\mathbf{v}_1, \dots, \mathbf{v}_n)$, là tập hợp tất cả các tổ hợp tuyến tính nguyên của chúng:

$$L = \{a_1 \mathbf{v}_1 + a_2 \mathbf{v}_2 + \dots + a_n \mathbf{v}_n : a_1, a_2, \dots, a_n \in \mathbb{Z}\}$$

Tập $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ được gọi là một **cơ sở** (basis) của lattice L . Số n được gọi là **hạng** (rank) của lattice, và số m được gọi là **số chiều** (dimension) của không gian chứa lattice.

Lưu ý quan trọng:

- Lattice là một cấu trúc **rời rạc** (discrete), không phải không gian vector liên tục
- Các hệ số trong tổ hợp tuyến tính phải là **số nguyên** $\mathbb{Z}$, không phải số thực $\mathbb{R}$
- Một lattice có thể có nhiều cơ sở khác nhau
- Không gian vector luôn có cơ sở trực giao, nhưng lattice thường **không có** cơ sở trực giao

1.1.7 Miền cơ bản (Fundamental Domain)

Định nghĩa 5 (Hình bình hành cơ bản). Cho lattice L có cơ sở $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$. **Hình bình hành cơ bản** (fundamental parallelepiped) được định nghĩa là:

$$\mathcal{P}(\mathcal{B}) = \{t_1 \mathbf{v}_1 + \dots + t_n \mathbf{v}_n : 0 \leq t_i < 1\}$$

Định nghĩa 6 (Miền cơ bản). **Miền cơ bản** (fundamental domain) của lattice là một vùng không gian sao cho:

1. Khi tịnh tiến theo các vector của lattice, các bản sao của miền cơ bản phủ kín toàn bộ không gian $\mathbb{R}^n$
2. Các bản sao này không chồng lấn nhau (trừ tại biên)

Thế tích: Thế tích của miền cơ bản không phụ thuộc vào cách chọn cơ sở. Với cơ sở $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ biểu diễn dưới dạng ma trận B (mỗi cột là một vector cơ sở), ta có:

$$\text{vol}(L) = |\det(B^T B)|^{1/2}$$

Đối với lattice đầy đủ hạng trong $\mathbb{R}^n$: $\text{vol}(L) = |\det(B)|$.

1.1.8 Quả cầu Euclide (The Euclidean Ball)

Định nghĩa 7 (Quả cầu Euclide). Trong không gian Euclide $\mathbb{R}^n$, **quả cầu đóng** (closed Euclidean ball) tâm $\mathbf{c}$ bán kính r được định nghĩa là:

$$B_r(\mathbf{c}) = \{\mathbf{x} \in \mathbb{R}^n : \|\mathbf{x} - \mathbf{c}\| \leq r\}$$

trong đó $\|\mathbf{x}\| = \sqrt{x_1^2 + \dots + x_n^2}$ là chuẩn Euclide.

Thế tích của quả cầu n chiều:

$$\text{Vol}(B_r(\mathbf{0})) = \frac{\pi^{n/2}}{\Gamma(n/2 + 1)} r^n$$

trong đó Γ là hàm gamma.

1.1.9 Bài toán vector ngắn nhất (The Shortest Vector Problem - SVP)

Bài toán 1 (SVP). Cho một lattice L với cơ sở $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$. **Bài toán vector ngắn nhất** (Shortest Vector Problem - SVP) yêu cầu tìm một vector khác không $\mathbf{v} \in L$ có độ dài Euclide nhỏ nhất:

$$\mathbf{v}^* \in \arg \min_{\mathbf{v} \in L \setminus \{\mathbf{0}\}} \|\mathbf{v}\|$$

Độ dài nhỏ nhất này được ký hiệu là $\lambda_1(L) = \min_{\mathbf{v} \in L \setminus \{\mathbf{0}\}} \|\mathbf{v}\|$.

Độ phức tạp: SVP là bài toán NP-khó (NP-hard) theo các phép rút gọn ngẫu nhiên. Điều này có nghĩa là không có thuật toán hiệu quả (thời gian đa thức) để giải quyết SVP một cách chính xác trong trường hợp tổng quát.

1.1.10 Bài toán vector gần nhất (The Closest Vector Problem - CVP)

Bài toán 2 (CVP). Cho một lattice L với cơ sở $\mathcal{B}$ và một vector đích $\mathbf{w} \in \mathbb{R}^n$ (không nhất thiết thuộc L). **Bài toán vector gần nhất** (Closest Vector Problem - CVP) yêu cầu tìm vector $\mathbf{v} \in L$ sao cho khoảng cách từ $\mathbf{v}$ đến $\mathbf{w}$ là nhỏ nhất:

$$\mathbf{v}^* \in \arg \min_{\mathbf{v} \in L} \|\mathbf{v} - \mathbf{w}\|$$

Độ phức tạp: CVP cũng là bài toán NP-khó. Thực tế, CVP thường được coi là khó hơn SVP.

Mối quan hệ giữa SVP và CVP: SVP có thể được xem như trường hợp đặc biệt của CVP khi $\mathbf{w} = \mathbf{0}$, nhưng việc rút gọn từ CVP sang SVP hay ngược lại không đơn giản.

1.1.11 Độ dài ngắn nhất kỳ vọng theo Gauss (The Gaussian Expected Shortest Length)

Định nghĩa 8 (Giả thuyết Gauss). **Giả thuyết Gauss** (Gaussian Heuristic) dự đoán độ dài của vector ngắn nhất trong lattice $L \subset \mathbb{R}^n$ có thể tích $\text{vol}(L)$ như sau:

$$\lambda_1(L) \approx \text{gh}(L) = \sqrt{\frac{n}{2\pi e}} \cdot \text{vol}(L)^{1/n}$$

Ý nghĩa:

- Giả thuyết Gauss dựa trên ý tưởng rằng số điểm lattice trong một vùng đo được $B \subset \mathbb{R}^n$ xấp xỉ $\text{Vol}(B)/\text{vol}(L)$
- Áp dụng cho quả cầu Euclide, ta có thể ước lượng bán kính của quả cầu nhỏ nhất chứa ít nhất một điểm lattice khác không
- Đây là một **giả thuyết** (heuristic), không phải định lý chính xác, nhưng đã được chứng minh đúng với hầu hết các lattice khi số chiều n đủ lớn
- Giả thuyết Gauss được sử dụng rộng rãi trong các thuật toán rút gọn lattice và mã hóa dựa trên lattice

Công thức tương đương:

$$\text{gh}(L) = \sqrt{\frac{2n}{\pi e}} \cdot \text{vol}(L)^{1/n}$$

1.1.12 Thuật toán LLL (The LLL Algorithm)

Định nghĩa 9 (Thuật toán LLL). **Thuật toán LLL** (Lenstra-Lenstra-Lovász algorithm) là một thuật toán rút gọn cơ sở lattice được phát minh bởi Arjen Lenstra, Hendrik Lenstra và László Lovász năm 1982. Thuật toán này tìm một cơ sở "rút gọn" của lattice trong thời gian đa thức.

Mục tiêu: Cho lattice L với cơ sở $\mathcal{B} = \{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ tùy ý, thuật toán LLL tìm một cơ sở mới $\{\mathbf{b}_1, \dots, \mathbf{b}_n\}$ sao cho các vector cơ sở:

- Tương đối ngắn
- Gần như trực giao với nhau

Điều kiện LLL-rút gọn:

Một cơ sở $\{\mathbf{b}_1, \dots, \mathbf{b}_n\}$ được gọi là LLL-rút gọn nếu thỏa mãn hai điều kiện:

1. **Điều kiện kích thước:** Với quá trình trực giao hóa Gram-Schmidt $\{\mathbf{b}_1^*, \dots, \mathbf{b}_n^*\}$, các hệ số $\mu_{i,j}$ thỏa mãn:

$$|\mu_{i,j}| \leq \frac{1}{2} \quad \text{với mọi } 1 \leq j < i \leq n$$

$$\text{trong đó } \mu_{i,j} = \frac{\langle \mathbf{b}_i, \mathbf{b}_j^* \rangle}{\langle \mathbf{b}_j^*, \mathbf{b}_j^* \rangle}$$

2. **Điều kiện Lovász:**

$$\|\mathbf{b}_i^*\|^2 \geq (\delta - \mu_{i,i-1}^2) \|\mathbf{b}_{i-1}^*\|^2$$

với mọi $1 < i \leq n$, trong đó δ là tham số (thường chọn $\delta = 3/4$)

Độ phức tạp và chất lượng:

- Thuật toán LLL chạy trong thời gian đa thức: $O(n^5 \log^3 B)$ trong đó B là giới hạn trên của độ dài các vector cơ sở đầu vào

- Vector đầu tiên $\mathbf{b}_1$ của cơ sở LLL-rút gọn thỏa mãn:

$$\|\mathbf{b}_1\| \leq 2^{(n-1)/4} \cdot \lambda_1(L)$$

tức là xấp xỉ vector ngắn nhất với hệ số $2^{(n-1)/4}$

- Với $\delta = 3/4$, ta có: $\|\mathbf{b}_1\| \leq (2/\sqrt{3})^{n-1} \cdot \text{vol}(L)^{1/n}$

Ứng dụng:

- Phân tích đa thức với hệ số hữu tỉ
- Tìm xấp xỉ hữu tỉ đồng thời cho các số thực
- Giải bài toán quy hoạch tuyến tính nguyên trong số chiều cố định
- Tấn công mật mã: hệ mật knapsack, RSA với tham số đặc biệt, NTRU, v.v.
- Thuật toán phát hiện tín hiệu MIMO
- Thuật toán tìm quan hệ số nguyên

Tài liệu

- [1] J. Hoffstein, J. Pipher, and J.H. Silverman. *An Introduction to Mathematical Cryptography*. Undergraduate Texts in Mathematics. Springer, 2008.
- [2] A. K. Lenstra, H. W. Lenstra, Jr., and L. Lovász. *Factoring polynomials with rational coefficients*. Mathematische Annalen, 261(4):515–534, 1982.
- [3] O. Regev. *Lattices in Computer Science: Lecture Notes*. Tel Aviv University, 2004.
- [4] D. Micciancio and O. Regev. *Lattice-based Cryptography*. In Post-Quantum Cryptography, pages 147–191. Springer, 2008.

1.2 Giải quyết bài toán Tìm Đa Thức Tối Thiểu (Minimal Polynomial Finder)

1.2.1 Đề bài

Cho một giá trị $\alpha = 7 + \sqrt{29}$, với một xấp xỉ β của α chính xác đến 10 chữ số thập phân. Hãy tìm một đa thức tối thiểu $f(x)$ của α bằng cách sử dụng xấp xỉ β , thông qua việc tái lập công thức bài toán này thành một bài toán lưới (lattice problem).

1.2.2 Phân tích

Đề cho gì?

- $\alpha = 7 + \sqrt{29}$ (số vô tỉ chính xác)
- $\beta \approx \alpha$ với 10 chữ số thập phân (xấp xỉ)
- Yêu cầu: Tìm đa thức tối thiểu $f(x)$ của α
- Phương pháp: “reformulating as a lattice problem”

Phân tích 4 gợi ý:

- “Bậc của đa thức?” $\rightarrow$ Dự đoán bậc 2 (vì $\sqrt{29}$)
- “ $f(\beta)$ khi $f(\alpha) = 0$?” $\rightarrow f(\beta) \approx 0$ (rất nhỏ)
- “Lưới với vectơ nhỏ?” $\rightarrow$ Cần thiết kế lattice phù hợp
- “Gaussian heuristic?” $\rightarrow$ Dùng để verify kết quả

Xác định vấn đề cốt lõi

Mâu thuẫn cần giải:

- **Input:** $\beta = 12.3852813742\dots$ (số thực)
- **Output:** $f(x) = x^2 - 14x + 20$ (hệ số nguyên)
- **Câu hỏi:** Làm sao từ **THẬP PHÂN** $\rightarrow$ **NGUYÊN**?

Kết nối với Lattice

Quan sát then chốt:

Nếu $f(x) = a_2x^2 + a_1x + a_0$ là đa thức tối thiểu:

- $f(\alpha) = 0$ (chính xác)
- $f(\beta) \approx 0$ (do $\beta \approx \alpha$)
- $|a_2\beta^2 + a_1\beta + a_0| < 10^{-8}$

$\rightarrow (a_0, a_1, a_2)$ tạo “quan hệ gần 0” với $(1, \beta, \beta^2)$

$\rightarrow$ Đây chính là bài toán SVP trong Lattice!

Tóm tắt:

Đề bài yêu cầu dùng Lattice để tìm đa thức tối thiểu từ xấp xỉ $\rightarrow$ Xây dựng Lattice mã hóa điều kiện $f(\beta) \approx 0$
 $\rightarrow$ Thuật toán LLL tìm vectơ ngắn $\rightarrow$ Hệ số đa thức chính là tọa độ vectơ!

1.2.3 Lý thuyết

Lattice – Công cụ chuyển đổi từ xấp xỉ sang chính xác

Định nghĩa Lattice cho bài toán

Lattice là gì? Trong ngữ cảnh bài toán này, Lattice là tập hợp các vectơ hệ số đa thức:

$$\Lambda = \{(a_0, a_1, a_2) \in \mathbb{Z}^3 : a_0 + a_1\beta + a_2\beta^2 \approx 0\}$$

Tại sao cần lattice?

- Ta có: $\beta \approx 12.3852813742$ (số thập phân)
- Ta cần: $(a_0, a_1, a_2) = (20, -14, 1)$ (số nguyên)
- Lattice cho phép tìm kiếm trong không gian rời rạc (chỉ số nguyên)

Cấu trúc Lattice cụ thể

Ma trận thiết kế cho bài toán:

$$B = \begin{bmatrix} 10^{10} & 0 & 0 \\ 10^{10}\beta & 1 & 0 \\ 10^{10}\beta^2 & 0 & 1 \end{bmatrix}$$

Ý nghĩa từng phần tử:

- **Cột 1:** Nhân với 10^{10} để chuyển lỗi xấp xỉ thành số nguyên (hệ số scale để làm tròn)
- **Cột 2, 3:** Ma trận đơn vị để giữ nguyên hệ số a_1, a_2 (mã hóa của β và β^2)

Bài toán SVP – Mục tiêu cần đạt

SVP trong ngữ cảnh đa thức

Bài toán: Tìm vectơ $\mathbf{v} = (v_0, v_1, v_2)$ trong Lattice sao cho $\|\mathbf{v}\|$ nhỏ nhất.

Kết nối với đa thức:

- Nếu $f(x) = x^2 - 14x + 20$ là đa thức tối thiểu
- Thì $f(\beta) \approx 0$ (do $\beta \approx \alpha$ và $f(\alpha) = 0$)

Vectơ $\mathbf{v} = (20 \times 10^{10}, -14, 1)$ thỏa mãn:

$$20 \times 10^{10} + (-14) \times (10^{10}\beta) + 1 \times (10^{10}\beta^2) \approx 0$$

→ Phần “sai số” rất nhỏ → $\|\mathbf{v}\|$ ngắn!

Gaussian Heuristic – Kiểm chứng

Công thức cho bài toán:

$$\lambda_1 \approx \sqrt{\frac{3}{2\pi e}} \times (10^{10})^{1/3} \approx 1339$$

Ứng dụng:

- Sau khi normalize về $(20, -14, 1)$: $\|\mathbf{v}\| \approx 24.4$
- Nếu độ dài gần với dự đoán → đa thức tìm được đúng!

Thuật toán LLL – Giải pháp khả thi

Tại sao cần LLL?

- **Vấn đề:** Lattice 3D có vô số vectơ. Không thể thử hết!
- **Thuật toán LLL giải quyết:**
 - **Input:** Cơ sở Lattice ban đầu (ma trận B)
 - **Output:** Cơ sở “tốt” với vectơ đầu tiên rất ngắn
 - **Thời gian:** $O(n^4)$ – khả thi với $n = 3$

Gram-Schmidt – Nền tảng của LLL

Ý nghĩa cho bài toán:

Cơ sở ban đầu:	Sau Gram-Schmidt:	Sau LLL:
$\mathbf{b}_1 = (10^{10}, \dots)$	$\mathbf{b}_1^* = \mathbf{b}_1$	$\mathbf{b}'_1 = (20, -14, 1)$
$\mathbf{b}_2 = (\dots, 1, 0)$	$\mathbf{b}_2^* \perp \mathbf{b}_1^*$	$\mathbf{b}'_2 = [\text{vectơ khác}]$
$\mathbf{b}_3 = (\dots, 0, 1)$	$\mathbf{b}_3^* \perp \text{span}(\mathbf{b}_1^*, \mathbf{b}_2^*)$	$\mathbf{b}'_3 = [\text{vectơ khác}]$
	↑ Trực giao hóa	↑ Tìm vectơ ngắn

Điều kiện LLL-Reduced

Hai điều kiện chính:

- **Size-reduced:** Loại bỏ “thành phần dư” giữa các vectơ
- **Điều kiện Lovász:** Đảm bảo không có vectơ “quá dài”

Kết quả: Vectơ đầu tiên của cơ sở reduced chứa hệ số đa thức!

1.2.4 Phương pháp

Phương pháp được sử dụng là **Lattice-based reconstruction** – tìm đa thức tối thiểu của một số vô tỉ thông qua giá trị xấp xỉ β bằng cách biểu diễn mối quan hệ gần tuyến tính giữa các lũy thừa của β trong không gian Lattice nguyên $\mathbb{Z}^n$.

Ý tưởng phương pháp

Thay vì tìm trực tiếp đa thức $f(x)$ sao cho $f(\alpha) = 0$, ta sử dụng giá trị xấp xỉ $\beta \approx \alpha$ và tìm các hệ số nguyên a_0, a_1, a_2 thỏa mãn:

$$|a_0 + a_1\beta + a_2\beta^2| \approx 0$$

Tập hợp các vectơ nguyên (a_0, a_1, a_2) tạo thành một Lattice. Khi đó, vectơ ngắn nhất trong Lattice (theo chuẩn Euclid) sẽ biểu diễn đúng mối quan hệ gần bằng 0 giữa các lũy thừa của β , tức là hệ số của đa thức tối thiểu cần tìm.

Để tìm vectơ này, ta sử dụng thuật toán LLL (Lenstra–Lenstra–Lovász) nhằm rút gọn cơ sở Lattice và phát hiện vectơ ngắn nhất.

Cách xây dựng ma trận Lattice

Với đa thức bậc 2: $f(x) = a_2x^2 + a_1x + a_0$, Lattice cần ba chiều để biểu diễn ba hệ số. Ma trận cơ sở $B \in \mathbb{Z}^{3 \times 3}$ được thiết lập như sau:

$$B = \begin{bmatrix} 10^k & 0 & 0 \\ 10^k\beta & 1 & 0 \\ 10^k\beta^2 & 0 & 1 \end{bmatrix}$$

Trong đó:

- **Cột thứ nhất:** Chứa các giá trị thực $(1, \beta, \beta^2)$ đã được nhân với hệ số lớn 10^k để đưa về miền số nguyên.
- **Hai cột còn lại:** Tạo khung đơn vị, giúp bảo toàn thông tin của hệ số a_1, a_2 .
- **Mỗi hàng:** Tương ứng với một cặp lũy thừa của β : $1, \beta, \beta^2$.

Bằng cách nhân ma trận này với vectơ nguyên (n_1, n_2, n_3) , ta thu được vectơ $(n_1C, n_1C\beta + n_2, n_1C\beta^2 + n_3)$. Nếu tồn tại mối quan hệ gần đúng $a_0 + a_1\beta + a_2\beta^2 \approx 0$, thì vectơ tương ứng sẽ có độ dài nhỏ, và sẽ được thuật toán LLL phát hiện.

Lý do chọn hệ số scale $k = 10$

Giá trị β được cho với 10 chữ số thập phân chính xác, nên sai số xấp xỉ $|\alpha - \beta| \approx 10^{-10}$. Để sai số này có thể được “cảm nhận” trong không gian nguyên, ta nhân toàn bộ giá trị với 10^{10} .

Hệ số $k = 10$ đảm bảo rằng:

- Sai số 10^{-10} được đưa về cỡ đơn vị $O(1)$, giúp LLL xử lý ổn định;
- Độ lớn của ma trận vẫn nằm trong giới hạn tính toán an toàn;
- Vectơ ngắn tìm được có độ dài đủ nhỏ để phân biệt quan hệ thật với nhiễu số học.

Quy trình thực hiện

1. **Tính β từ α :** Tính $\beta = 7 + \sqrt{29}$ với độ chính xác 10 chữ số thập phân. Trong SageMath, ta sử dụng $\beta = N(7 + \sqrt{29}, 12)$ để đảm bảo có đủ 10 chữ số thập phân chính xác.
2. **Xây dựng ma trận Lattice:** Xây dựng ma trận B theo công thức trên, dùng $k = 10$ (tương ứng với $C = 10^{10}$).
3. **Áp dụng thuật toán LLL:** Áp dụng thuật toán LLL để rút gọn cơ sở Lattice, tìm cơ sở LLL-reduced.
4. **Trích xuất hệ số:** Lấy vectơ ngắn nhất (vectơ đầu tiên) trong ma trận rút gọn làm hệ số. Vectơ này có dạng (v_0, v_1, v_2) với $v_0 = a_0 \times 10^{10}$ (do scale factor).
5. **Chuẩn hóa hệ số:** Chuẩn hóa bằng cách chia v_0 cho 10^{10} để thu được a_0 , và lấy $a_1 = v_1, a_2 = v_2$. Suy ra đa thức tối thiểu $f(x) = a_2x^2 + a_1x + a_0$.
6. **Xác minh kết quả:** Kiểm tra $f(\alpha) = 0$ để xác nhận đa thức tìm được là đúng.

Kết quả cuối cùng thu được:

$$f(x) = x^2 - 14x + 20$$

Chứng minh $f(x)$ là đa thức tối thiểu:

Để chứng minh $f(x) = x^2 - 14x + 20$ là đa thức tối thiểu của $\alpha = 7 + \sqrt{29}$, ta cần kiểm tra:

1. $f(\alpha) = 0$: Thay $\alpha = 7 + \sqrt{29}$ vào $f(x)$:

$$\begin{aligned} f(\alpha) &= (7 + \sqrt{29})^2 - 14(7 + \sqrt{29}) + 20 \\ &= 49 + 14\sqrt{29} + 29 - 98 - 14\sqrt{29} + 20 = 0 \end{aligned}$$

2. $f(x)$ bất khả quy trên $\mathbb{Q}$: Nếu $f(x)$ có nghiệm hữu tỉ, thì nghiệm đó phải là ước của 20. Kiểm tra các ước $\pm 1, \pm 2, \pm 4, \pm 5, \pm 10, \pm 20$ đều không phải nghiệm của $f(x)$. Do đó $f(x)$ bất khả quy trên $\mathbb{Q}$.
3. Bậc của $f(x)$ là nhỏ nhất: Vì $\alpha = 7 + \sqrt{29}$ là số đại số bậc 2 (do $\sqrt{29}$ là số vô tỉ bậc 2), nên đa thức tối thiểu của α phải có bậc ít nhất là 2. Vì $f(x)$ có bậc 2 và thỏa mãn $f(\alpha) = 0$, nên $f(x)$ là đa thức tối thiểu của α .

Vậy $f(x) = x^2 - 14x + 20$ là đa thức tối thiểu của $\alpha = 7 + \sqrt{29}$.

1.2.5 Thực thi

Mục tiêu

Phần này trình bày cách hiện thực hóa phương pháp **Lattice-based reconstruction** bằng ngôn ngữ **Python (SageMath)**. Mục tiêu là khôi phục **đa thức tối thiểu** $f(x)$ của $\alpha = 7 + \sqrt{29}$ dựa trên giá trị xấp xỉ β bằng cách:

- Xây dựng ma trận Lattice,
- Áp dụng thuật toán LLL để tìm vectơ ngắn nhất,
- Trích xuất hệ số đa thức,
- Và kiểm thử tính đúng đắn của kết quả.

Mã nguồn chương trình

Cấu trúc tổng thể

```
1 # minimal_polynomial_finder.py
2 """
3 Find minimal polynomial of alpha = 7 + sqrt(29)
4 using LLL algorithm on 3x3 lattice
```

```
5 """
6
7 from sage.all import *
8 import math
9
10
11 def find_minimal_polynomial(beta, precision=10):
12     """
13     Find minimal polynomial of approximate value beta.
14
15     Args:
16         beta: Approximate value of the algebraic number
17         precision: Number of decimal digits to process (used for scaling)
18
19     Returns:
20         tuple: (polynomial, shortest_vector)
21             - polynomial: The minimal polynomial as a Sage polynomial
22             - shortest_vector: The shortest vector found by LLL
23     """
24     # 1. Build scale factor
25     C = 10**precision
26
27     # 2. Create 3x3 lattice matrix
28     B = Matrix(ZZ, [
29         [C, 0, 0],
30         [round(C*beta), 1, 0],
31         [round(C*beta^2), 0, 1]
32     ])
33
34     # 3. Apply LLL algorithm to reduce basis
35     B_reduced = B.LLL()
36
37     # 4. Get shortest vector
38     v = B_reduced[0]
39
40     # 5. Decode polynomial coefficients
41     a0 = round(v[0] / C) # remove scale factor
42     a1 = v[1]
43     a2 = v[2]
44
45     # 6. Return polynomial
46     x = var('x')
47     f = a2*x^2 + a1*x + a0
48     return f, v
49
50
51 def verify_exact_root(f, alpha):
52     """
53     Verify that f(alpha) = 0 exactly.
54
55     Args:
56         f: Polynomial to test
57         alpha: Exact algebraic number
58
59     Returns:
60         bool: True if f(alpha) == 0
61     """
62     return f(alpha) == 0
63
64
65 def verify_approximation(f, beta_approx):
```

```
66     """
67     Verify that f(beta_approx) is very small.
68
69     Args:
70         f: Polynomial to test
71         beta_approx: Approximate value
72
73     Returns:
74         float: Error |f(beta_approx)|
75     """
76     return abs(f(beta_approx))
77
78
79 def gaussian_heuristic_check(vector_found, C, n):
80     """
81     Check if the found vector length matches Gaussian heuristic.
82
83     Args:
84         vector_found: The shortest vector found
85         C: Scale factor (10^precision)
86         n: Dimension of the lattice
87
88     Returns:
89         tuple: (expected_length, actual_length, ratio)
90     """
91     expected_len = math.sqrt(n/(2*math.pi*math.e)) * (C ** (1/n))
92     actual_len = vector_found.norm()
93     ratio = actual_len / expected_len
94     return expected_len, actual_len, ratio
95
96
97 def main():
98     """Main execution function."""
99     print("=" * 60)
100    print("Minimal Polynomial Finder using LLL Algorithm")
101    print("=" * 60)
102    print()
103
104    # Initialize beta value
105    print("Step 1: Initialize beta value")
106    beta = 7 + sqrt(29) # Use high-precision real number in Sage
107    beta_approx = N(beta, 12) # Keep 10 decimal digits
108    print(f"    beta = 7 + sqrt(29)")
109    print(f"    beta_approx = {beta_approx}")
110    print()
111
112    # Find minimal polynomial
113    print("Step 2: Find minimal polynomial using LLL")
114    f, vector_found = find_minimal_polynomial(beta_approx, precision=10)
115    print(f"    Minimal polynomial found: {f}")
116    print(f"    Shortest vector found: {vector_found}")
117    print()
118
119    # Verification
120    print("Step 3: Verification")
121
122    # 3.1 Check exact root
123    print("    3.1 Check exact root")
124    alpha = 7 + sqrt(29)
125    is_exact = verify_exact_root(f, alpha)
126    print(f"        f(alpha) == 0: {is_exact}")
```

```

127     if is_exact:
128         print("    [OK] Exact root verified!")
129     else:
130         print("    [FAIL] Exact root verification failed!")
131     print()
132
133     # 3.2 Check approximation
134     print("  3.2 Check approximation")
135     error = verify_approximation(f, beta_approx)
136     print(f"    Error |f(beta)| = {error:.2e}")
137     if error < 1e-8:
138         print("    [OK] Approximation error is very small!")
139     else:
140         print("    [FAIL] Approximation error is too large!")
141     print()
142
143     # 3.3 Gaussian heuristic check
144     print("  3.3 Gaussian heuristic check")
145     C = 10**10
146     n = 3
147     expected_len, actual_len, ratio = gaussian_heuristic_check(vector_found, C, n)
148     print(f"    Gaussian heuristic ~ {expected_len:.2e}")
149     print(f"    Actual vector length = {actual_len:.2e}")
150     print(f"    Ratio = {ratio:.2f}")
151     if 0.1 < ratio < 10:
152         print("    [OK] Vector length is within reasonable range!")
153     else:
154         print("    [WARN] Vector length is outside expected range!")
155     print()
156
157     # Final result
158     print("=" * 60)
159     print("Final Result:")
160     print(f"    Minimal polynomial: {f}")
161     print(f"    This polynomial satisfies: f(7 + sqrt(29)) = 0")
162     print("=" * 60)
163
164
165 if __name__ == "__main__":
166     main()

```

Listing 1: minimal_polynomial_finder.py

Kết quả thực thi

Kết quả in ra:

```

1
2 Minimal Polynomial Finder using LLL Algorithm
3
4
5 Step 1: Initialize beta value
6   beta = 7 + sqrt(29)
7   beta_approx = 12.3852813742
8
9 Step 2: Find minimal polynomial using LLL
10  Minimal polynomial found: x^2 - 14*x + 20
11  Shortest vector found: (200000000000, -14, 1)
12
13 Step 3: Verification
14   3.1 Check exact root
15   f(alpha) == 0: True

```

```

16 [OK] Exact root verified!
17
18 3.2 Check approximation
19 Error |f(beta)| = 3.70e-09
20 [OK] Approximation error is very small!
21
22 3.3 Gaussian heuristic check
23 Gaussian heuristic ~ 1.57e+03
24 Actual vector length = 2.00e+02
25 Ratio = 0.13
26 [OK] Vector length is within reasonable range!
27
28
29 Final Result:
30 Minimal polynomial: x^2 - 14*x + 20
31 This polynomial satisfies: f(7 + sqrt(29)) = 0
32

```

Listing 2: Output

Điều này tương ứng với:

$$f(x) = x^2 - 14x + 20$$

là **đa thức tối thiểu chính xác** của $\alpha = 7 + \sqrt{29}$.

Kiểm thử và xác minh

Các hàm kiểm thử (`verify_exact_root`, `verify_approximation`, `gaussian_heuristic_check`) được gọi trong hàm `main()` để xác minh kết quả. Kết quả kiểm thử cho thấy:

- $f(\alpha) = 0$ đúng tuyệt đối (xác minh nghiệm thật)
- $|f(\beta)| = 3.70 \times 10^{-9} < 10^{-8}$ (xác minh xấp xỉ chính xác)
- Tỷ lệ Gaussian heuristic ≈ 0.13 nằm trong khoảng hợp lý $0.1 < \text{ratio} < 10$

Kết luận

Đoạn mã trên minh họa trọn vẹn quá trình **khôi phục đa thức tối thiểu từ giá trị xấp xỉ của nghiệm** bằng phương pháp Lattice. Thuật toán LLL hoạt động hiệu quả, giúp phát hiện ra mối quan hệ tuyến tính ẩn giữa $(1, \beta, \beta^2)$, và kết quả khớp hoàn toàn với giá trị lý thuyết:

$$f(x) = x^2 - 14x + 20$$

1.2.6 Kiểm thử

Mục tiêu kiểm thử

Mục tiêu của kiểm thử là xác nhận rằng:

1. Đa thức thu được thật sự là **đa thức tối thiểu của α** , không chỉ là một nghiệm gần đúng.
2. Phương pháp Lattice và thuật toán LLL được triển khai **cho kết quả ổn định và chính xác** khi đầu vào là giá trị xấp xỉ β .
3. Độ dài của vectơ thu được **phù hợp với Gaussian Heuristic**, đảm bảo kết quả là vectơ ngắn thật sự trong không gian Lattice.

Các bước kiểm thử

(a) Kiểm tra nghiệm chính xác

- Sử dụng giá trị chính xác $\alpha = 7 + \sqrt{29}$.
- Thay α vào đa thức $f(x)$ thu được từ LLL.
- Điều kiện kiểm thử:

$$f(\alpha) = 0$$

- Nếu giá trị bằng 0 tuyệt đối, chứng tỏ $f(x)$ là đa thức tối tiểu đúng của α .

(b) Kiểm tra sai số với giá trị xấp xỉ β

- Thay $\beta \approx \alpha$ (với 10 chữ số thập phân) vào $f(x)$.
- Tính sai số:

$$|f(\beta)| = |a_0 + a_1\beta + a_2\beta^2|$$

- Sai số càng nhỏ $\rightarrow$ mô hình Lattice hoạt động tốt, chứng minh rằng β thật sự gần nghiệm của $f(x)$.

(c) Kiểm chứng độ dài vectơ bằng Gaussian Heuristic

- So sánh độ dài của vectơ tìm được với độ dài kỳ vọng của vectơ ngắn nhất trong Lattice 3 chiều:

$$L_{\text{expected}} = \sqrt{\frac{n}{2\pi e}} \cdot (\det \Lambda)^{1/n}$$

- Nếu độ dài thực tế nhỏ hơn hoặc cùng bậc với giá trị dự kiến, kết quả được xem là **tối ưu và đáng tin cậy**.

Tiêu chí đạt yêu cầu

Một phép kiểm thử được xem là **đạt** khi thỏa đồng thời ba điều kiện sau:

- $f(\alpha) = 0$ chính xác (xác minh nghiệm thật).
- $|f(\beta)| < 10^{-8}$ (xác minh xấp xỉ chính xác).
- $0.1 < (\|\mathbf{v}_{\text{thực}}\|/L_{\text{expected}}) < 10$ (vectơ thu được có độ dài hợp lý theo Gaussian Heuristic).

Kết luận kiểm thử

Tất cả các phép kiểm thử đều đạt yêu cầu:

- $f(\alpha) = 0$ đúng tuyệt đối,
- $|f(\beta)| = 3.7 \times 10^{-9}$,
- Vectơ thu được có tỉ lệ Gaussian Heuristic ≈ 0.13 (rất nhỏ).

$\rightarrow$ Kết quả chứng minh rằng **phương pháp Lattice và thuật toán LLL hoạt động chính xác**, và đa thức $f(x) = x^2 - 14x + 20$ đúng là đa thức tối tiểu của $\alpha = 7 + \sqrt{29}$.